

PROJECT REPORT

EXPERIMENT - 17

PROJECT TITLE

Predictive Maintenance and Machine Health
Monitoring System

Submitted by

MUTHU KRISHNA P - 24MER033

PRIYANTH C S - 24MER044

SRI RAMAN G - 24MEL068

PRADEEP P - 24MER041

Predictive Maintenance and Machine Health Monitoring System

1. Project Overview

The **Predictive Maintenance and Machine Health Monitoring System** is an IoT-based system designed to continuously monitor the health and operating condition of industrial machines. The system collects important machine parameters such as **temperature, vibration, current, voltage, speed, and operating hours** and analyzes them to identify abnormal operating conditions.

Traditional maintenance methods are generally based on either fixed schedules or repairing equipment after a failure occurs. Predictive maintenance is more efficient because it uses real-time machine data to identify early signs of wear, overheating, imbalance, misalignment, overloading, or other faults.

The proposed system uses an **ESP8266 NodeMCU** as the main controller. Sensors connected to the controller collect machine-health parameters. The ESP8266 processes the sensor data, compares it with predefined limits, displays the machine condition on an OLED display, activates an alarm when necessary, and transmits the data wirelessly to a cloud or MQTT monitoring platform.

2. Problem Statement

Industrial machines are subjected to continuous mechanical and electrical stress during operation. Components such as bearings, motors, shafts, gears, belts, and electrical windings can gradually deteriorate.

In conventional maintenance systems, machines are often inspected manually or serviced according to a fixed maintenance schedule. This approach has several limitations.

A machine may fail **before its scheduled maintenance**, while another machine may be serviced even though it is still operating normally. Unexpected equipment failure can result in:

- Production downtime
- Increased maintenance costs
- Equipment damage
- Reduced productivity

- Safety risks
- Loss of production

3. Proposed Solution

The proposed solution is an **IoT-based Predictive Maintenance and Machine Health Monitoring System** using an ESP8266 NodeMCU.

Sensors are installed on or near the machine to continuously collect operating data.

Main sensors

- **Temperature sensor** – monitors machine temperature.
- **Vibration sensor/accelerometer** – detects abnormal mechanical vibration.
- **Current sensor** – monitors electrical load.
- **Voltage sensor** – monitors supply voltage.
- **RPM sensor** – measures rotational speed, if required.

The ESP8266 processes the collected data and determines the machine-health condition.

4. System Architecture

The system architecture is divided into several layers.

4.1 Machine Layer

The industrial machine is the equipment being monitored.

Examples include:

- Motor
- Pump
- Fan
- Compressor
- Conveyor
- Gearbox

4.2 Sensor Layer

Sensors collect information from the machine.

Typical measurements include:

- Temperature
- Vibration
- Current
- Voltage
- RPM

4.3 Processing Layer

The ESP8266 NodeMCU acts as the central processing unit.

Its functions include:

- Reading sensor values
- Filtering measurements
- Calculating machine-health indicators
- Comparing readings with limits
- Generating warnings
- Controlling the display and buzzer
- Sending data wirelessly

4.4 Communication Layer

The ESP8266 uses Wi-Fi to transmit the machine data.

4.5 Cloud Layer

The data can be transmitted to an MQTT broker or IoT cloud platform.

4.6 Monitoring Layer

A dashboard can display:

- Current temperature
- Vibration level

- Current consumption
- Voltage
- RPM
- Machine status
- Health score
- Historical trends

4.7 Maintenance Layer

Maintenance personnel can use the collected information to determine when an inspection or service is required.

5. Circuit Table

The following connection table is suitable for a college-level prototype using ESP8266.

Component	Pin/Output	ESP8266 Connection	Purpose
DS18B20 Temperature Sensor	DATA	D3 / GPIO0	Temperature
DS18B20	VCC	3.3V	Power
DS18B20	GND	GND	Ground
SW-420 Vibration Sensor	DO	D5 / GPIO14	Vibration detection
SW-420	VCC	3.3V	Power
SW-420	GND	GND	Ground
Current Sensor	Analog Output	External ADC	Current measurement
Voltage Sensor	Analog Output	External ADC	Voltage measurement

Component	Pin/Output	ESP8266 Connection	Purpose
RPM Sensor	Digital Output	D6 / GPIO12	Speed measurement
OLED	SDA	D2 / GPIO4	I ² C data
OLED	SCL	D1 / GPIO5	I ² C clock
OLED	VCC	3.3V	Power
OLED	GND	GND	Ground
Buzzer	Signal	D7 / GPIO13	Alarm

External ADC

The ESP8266 has only one built-in ADC input. Therefore, for multiple analog measurements such as current and voltage, an **ADS1115 external ADC** is recommended.

7. Algorithm

Step-by-step algorithm

Step 1: Start the system.

Step 2: Initialize ESP8266.

Step 3: Initialize temperature sensor.

Step 4: Initialize vibration sensor.

Step 5: Initialize current and voltage sensors.

Step 6: Initialize RPM sensor if available.

Step 7: Initialize OLED and buzzer.

Step 8: Connect to Wi-Fi.

Step 9: Read all machine parameters.

Step 10: Filter and process the sensor readings.

Step 11: Calculate machine-health indicators.

Step 12: Compare sensor values against predefined limits.

Step 13: Calculate the machine-health status.

Step 14: Display the machine condition.

Step 15: Activate the buzzer if necessary.

Step 16: Send data to the cloud/MQTT server.

Step 17: Store historical data.

Step 18: Analyze trends.

Step 19: Repeat continuously

8. Coding

Below is a prototype Arduino IDE program for the ESP8266.

```
#include <ESP8266WiFi.h>
```

```
#include <DHT.h>
```

```
#include "Adafruit_MQTT.h"
```

```
#include "Adafruit_MQTT_Client.h"
```

```
// =====
```

```
// WIFI
```

```
// =====
```

```
#define WIFI_SSID "Pradeep"
```

```
#define WIFI_PASSWORD "12345678"
```

```
// =====
```

```
// ADAFRUIT IO
```

```
// =====
```

```
#define AIO_SERVER "io.adafruit.com"
```

```
#define AIO_SERVERPORT 1883
```

```
#define AIO_USERNAME "muthumkt"
```

```
#define AIO_KEY "aio_DhKT47h46zwUYr2nRHBdifKDJtW0"
```

```
// =====
```

```
// PINS
```

```
// =====
```

```
#define VIBRATION_PIN D5
```

```
#define DHT_PIN D7
```

```
#define ACS_PIN A0
```

```
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
// =====
```

```
// LIMITS
```

```
// =====
```

```
float MAX_CURRENT = 10.0;
```

```
float MAX_TEMPERATURE = 60.0;
```

```
// =====
```

```
// ACS712
```



```
// =====
```

```
float ACS_SENSITIVITY = 0.100;
```

```
float ACS_ZERO_VOLTAGE = 0.50;
```

```
// =====
```

```
// WIFI + MQTT
```

```
// =====
```

```
WiFiClient client;
```

```
Adafruit_MQTT_Client mqtt(
```

```
  &client,
```

```
  AIO_SERVER,
```

```
  AIO_SERVERPORT,
```

```
  AIO_USERNAME,
```

```
  AIO_KEY
```

```
);
```

```
// =====
```

```
// ADAFRUIT FEEDS
```

```
// =====
```

```
Adafruit_MQTT_Publish vibrationFeed =
```

```
  Adafruit_MQTT_Publish(
```

```
    &mqtt,
```

```
AIO_USERNAME "/feeds/vibration"  
);
```

```
Adafruit_MQTT_Publish temperatureFeed =  
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/temperature"  
);
```

```
Adafruit_MQTT_Publish humidityFeed =  
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/humidity"  
);
```

```
Adafruit_MQTT_Publish currentFeed =  
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/current"  
);
```

```
Adafruit_MQTT_Publish statusFeed =  
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/machine-status"  
);
```

```
// =====  
// WIFI CONNECTION  
// =====  
  
void connectWiFi()  
{  
  if (WiFi.status() == WL_CONNECTED)  
  {  
    return;  
  }  
  
  Serial.println();  
  Serial.println("Connecting to WiFi...");  
  
  WiFi.mode(WIFI_STA);  
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);  
  
  int count = 0;  
  
  while (WiFi.status() != WL_CONNECTED && count < 30)  
  {  
    delay(500);  
    Serial.print(".");  
    count++;  
  }  
}
```

```
Serial.println();
```

```
if (WiFi.status() == WL_CONNECTED)
```

```
{
```

```
    Serial.println("WiFi Connected!");
```

```
    Serial.print("IP Address: ");
```

```
    Serial.println(WiFi.localIP());
```

```
}
```

```
else
```

```
{
```

```
    Serial.println("WiFi Connection Failed!");
```

```
}
```

```
}
```

```
// =====
```

```
// ADAFRUIT IO CONNECTION
```

```
// =====
```

```
bool connectMQTT()
```

```
{
```

```
    if (mqtt.connected())
```

```
    {
```

```
        return true;
```

```
}
```

```
Serial.println("Connecting to Adafruit IO...");

int8_t ret = mqtt.connect();

if (ret == 0)
{
    Serial.println("Adafruit IO Connected!");
    return true;
}

Serial.print("MQTT Error Code: ");
Serial.println(ret);

Serial.print("MQTT Error: ");
Serial.println(mqtt.connectErrorString(ret));

mqtt.disconnect();

return false;
}

// =====
// READ ACS712
// =====

float readCurrent()
```

```
{  
    long total = 0;  
  
    for (int i = 0; i < 100; i++)  
    {  
        total += analogRead(ACS_PIN);  
        delay(2);  
    }  
  
    float averageADC = total / 100.0;  
  
    // NodeMCU A0 assumed 0-1V  
    float sensorVoltage =  
        (averageADC / 1023.0) * 1.0;  
  
    float current =  
        (sensorVoltage - ACS_ZERO_VOLTAGE)  
        / ACS_SENSITIVITY;  
  
    if (current < 0)  
    {  
        current = -current;  
    }  
  
    if (current < 0.05)  
    {
```

```
        current = 0;
    }

    return current;
}

// =====
// SETUP
// =====

void setup()
{
    Serial.begin(115200);

    pinMode(VIBRATION_PIN, INPUT);

    dht.begin();

    delay(2000);

    connectWiFi();
    connectMQTT();

    Serial.println();
    Serial.println("=====");
    Serial.println(" PREDICTIVE MAINTENANCE SYSTEM");
```

```

Serial.println(" MACHINE HEALTH MONITORING");
Serial.println("=====");
Serial.println("SW-420    : READY");
Serial.println("DHT11     : READY");
Serial.println("ACS712    : READY");
Serial.println("WiFi      : READY");
Serial.println("Adafruit IO : READY");
Serial.println("=====");
}

// =====
// LOOP
// =====

void loop()
{
    // -----
    // WIFI
    // -----

    if (WiFi.status() != WL_CONNECTED)
    {
        connectWiFi();
    }

    // -----

```



```
// MQTT
// -----

if (!mqtt.connected())
{
    connectMQTT();
}

// -----
// READ VIBRATION
// -----

int vibrationValue =
    digitalRead(VIBRATION_PIN);

bool vibrationDetected =
    (vibrationValue == HIGH);

// -----
// READ DHT11
// -----

float temperature =
    dht.readTemperature();

float humidity =
```

```
dht.readHumidity();

// -----
// SENSOR ERROR
// -----

if (isnan(temperature) || isnan(humidity))
{
    Serial.println("DHT11 SENSOR ERROR");
    delay(2000);
    return;
}

// -----
// READ CURRENT
// -----

float current =
    readCurrent();

// -----
// MACHINE HEALTH
// -----

bool warning = false;
```

```
if (vibrationDetected)
```

```
{
```

```
    warning = true;
```

```
}
```

```
if (temperature >= MAX_TEMPERATURE)
```

```
{
```

```
    warning = true;
```

```
}
```

```
if (current >= MAX_CURRENT)
```

```
{
```

```
    warning = true;
```

```
}
```

```
const char* machineStatus;
```

```
if (warning)
```

```
{
```

```
    machineStatus = "WARNING";
```

```
}
```

```
else
```

```
{
```

```
    machineStatus = "NORMAL";
```

```
}
```

```
// -----  
// SERIAL MONITOR  
// -----  
  
Serial.println();  
Serial.println("-----");  
  
Serial.print("Vibration  :");  
  
if (vibrationDetected)  
{  
    Serial.println("DETECTED");  
}  
else  
{  
    Serial.println("NORMAL");  
}  
  
Serial.print("Temperature :");  
Serial.print(temperature, 1);  
Serial.println(" C");  
  
Serial.print("Humidity  :");  
Serial.print(humidity, 1);  
Serial.println(" %");
```

```
Serial.print("Current   : ");
```

```
Serial.print(current, 2);
```

```
Serial.println(" A");
```

```
Serial.print("Machine   : ");
```

```
Serial.println(machineStatus);
```

```
// -----
```

```
// SEND VIBRATION
```

```
// -----
```

```
if (mqtt.connected())
```

```
{
```

```
    int vibrationStatus = vibrationDetected ? 1 : 0;
```

```
    if (vibrationFeed.publish(vibrationStatus))
```

```
    {
```

```
        Serial.println("Vibration   : SENT");
```

```
    }
```

```
    else
```

```
    {
```

```
        Serial.println("Vibration   : FAILED");
```

```
    }
```

```
// -----
```

```
// SEND TEMPERATURE
```

```
// -----  
  
if (temperatureFeed.publish(temperature))  
{  
    Serial.println("Temperature : SENT");  
}  
else  
{  
    Serial.println("Temperature : FAILED");  
}  
  
// -----  
// SEND HUMIDITY  
// -----  
  
if (humidityFeed.publish(humidity))  
{  
    Serial.println("Humidity  : SENT");  
}  
else  
{  
    Serial.println("Humidity  : FAILED");  
}  
  
// -----  
// SEND CURRENT
```

```
// -----
```

```
if (currentFeed.publish(current))  
{  
    Serial.println("Current    : SENT");  
}  
else  
{  
    Serial.println("Current    : FAILED");  
}
```

```
// -----
```

```
// SEND MACHINE STATUS
```

```
// -----
```

```
if (statusFeed.publish(machineStatus))  
{  
    Serial.println("Machine    : SENT");  
}  
else  
{  
    Serial.println("Machine    : FAILED");  
}  
}  
else  
{
```

```
Serial.println("Adafruit IO : NOT CONNECTED");  
}
```

```
Serial.println("-----");
```

```
delay(5000);  
}
```

9. System Working

The system starts when power is supplied to the ESP8266. The controller initializes the sensors and OLED display and establishes a Wi-Fi connection.

The sensors continuously monitor the machine.

Temperature Monitoring

The temperature sensor measures the machine temperature.

A rising temperature can indicate:

- Excessive load
- Poor ventilation
- Bearing problems
- Lubrication problems
- Electrical winding problems

The system generates a warning when the temperature exceeds the defined limit.

Vibration Monitoring

Vibration is an important machine-health indicator.

Increasing vibration can indicate:

- Bearing wear
- Shaft misalignment
- Rotor imbalance

- Loose mechanical components
- Gear problems

The system detects abnormal vibration and changes the machine status.

Current Monitoring

Current consumption is monitored to identify electrical abnormalities.

An increase in current may indicate:

- Excessive mechanical load
- Motor problems
- Bearing friction

10. Bill of Materials

S.No.	Component	Quantity	Approx. Cost
1	ESP8266 NodeMCU	1	₹300–₹500
2	DS18B20 Temperature Sensor	1	₹100–₹200
3	Vibration Sensor / Accelerometer	1	₹50–₹500
4	Current Sensor	1	₹150–₹500
5	Voltage Sensor	1	₹100–₹300
6	RPM/Hall Sensor	1	₹100–₹250
7	ADS1115 ADC	1	₹150–₹300
8	OLED Display	1	₹150–₹300
9	Buzzer	1	₹10–₹30
10	Breadboard	1	₹80–₹150
11	Jumper Wires	1 set	₹50–₹150
12	Resistors	1 set	₹20–₹50
13	USB Cable	1	₹50–₹150
14	Power Supply	1	₹100–₹300

Estimated prototype cost

Approximately ₹1,410–₹3,680, depending on the selected sensors and components.

Industrial-grade vibration, current, voltage, and temperature sensors will increase the cost.

11. Prototype Implementation

The prototype can be implemented using a small DC motor or laboratory motor to represent the industrial machine.

Step 1 – Controller Setup

The ESP8266 NodeMCU is mounted on the breadboard and supplied with regulated power.

Step 2 – Temperature Sensor

The temperature sensor is placed near the motor or monitored machine surface.

Step 3 – Vibration Sensor

The vibration sensor is attached firmly to the motor housing.

For a better prototype, an accelerometer can be used to obtain vibration measurements.

Step 4 – Current Sensor

The current sensor is installed in the motor supply circuit according to its rated measurement method.

Step 5 – Voltage Sensor

A properly isolated and scaled voltage-sensing circuit is connected to measure the supply voltage.

Step 6 – RPM Sensor

A Hall-effect sensor or optical sensor can be positioned to detect shaft rotation.

Step 7 – OLED

The OLED displays the real-time machine parameters.

Step 8 – Buzzer

The buzzer provides an audible warning when the machine condition becomes abnormal.

Step 9 – Software

The program is uploaded to the ESP8266 using Arduino IDE.

Step 10 – Testing

The prototype can be tested under different conditions.

Test	Expected Result
Normal motor operation	NORMAL
Increased temperature	WARNING
Excessive vibration	WARNING
Excessive current	WARNING
Abnormal voltage	WARNING
Multiple simultaneous faults	CRITICAL

12. Results & Performance Analysis

The proposed system provides continuous monitoring of machine-health parameters and can identify abnormal conditions before they develop into major failures.

Expected results

Parameter	Normal	Warning	Critical
Temperature	Normal	High	Very high
Vibration	Low	Increased	Excessive
Current	Normal	High	Very high
Voltage	Normal	Abnormal	Severely abnormal
RPM	Normal	Deviation	Major deviation
Health Score	>70%	40–70%	<40%

Temperature Analysis

Temperature monitoring provides an early indication of overheating. A gradual increase in temperature can be tracked over time to identify potential machine problems.

Vibration Analysis

Vibration is one of the most useful parameters for mechanical condition monitoring. Increasing vibration can indicate mechanical wear or imbalance.

Conclusion

The **Predictive Maintenance and Machine Health Monitoring System** provides an effective IoT-based approach for monitoring industrial equipment. By continuously measuring temperature, vibration, current, voltage, and rotational speed, the system can identify abnormal operating conditions at an early stage.

The ESP8266 acts as the central controller, processes the sensor information, calculates a simplified machine-health status, displays the information locally, generates alarms, and provides wireless connectivity for remote monitoring.

The system can be further developed into a more advanced predictive-maintenance platform by collecting historical data and applying **trend analysis, FFT-based vibration analysis, machine-learning models, anomaly detection, and remaining-useful-life estimation**.

OUTPUT:

```
File Edit Sketch Tools Help
code2

#include <ESP8266WiFi.h>
#include <DHT.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// =====
// WIFI
// =====

#define WIFI_SSID "Pradeep"
#define WIFI_PASSWORD "12345678"

// =====
// ADAFRUIT IO
// =====

#define AIO_SERVER "io.adafruit.com"
#define AIO_SERVERPORT 1883

#define AIO_USERNAME "muthumkt"
#define AIO_KEY "aio_DhKT47h46zwUYr2nRHBd1fKDJLW0"

// =====
// PINS
// =====

#define VIBRATION_PIN D5
#define DHT_PIN D7
#define ACS_PIN A0

#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

Leaving...
Hard resetting via RTS pin...
```

```
COM7
Machine : SENT

Vibration : DETECTED
Temperature : 36.2 C
Humidity : 47.4 %
Current : 2.78 A
Machine : WARNING
Vibration : SENT
Temperature : SENT
Humidity : SENT
Current : SENT
Machine : SENT
```



